

可贖回CMS連結每日區間計息 技術文件

債券部

承辦人員：黃浩軒

目錄

目錄.....	2
一、 可贖回 CMS 連結每日區間計息債券	3
1、 契約內容.....	3
二、 LMM 利率模擬.....	5
1、 LMM 模型	5
三、 LMM 模型進行利率模擬.....	6
1、 CCDRA 產品假設.....	6
2、 區間計息內插法.....	7
3、 最小平方蒙地卡羅法(LEAST SQUARE MONTE CARLO SIMULATION METHOD)	7
四、 QUANT LIB 實作程式流程說明	9
1、 利率期間結構實作.....	9
2、 LMM 模型參數 CALIBRATION	9
3、 LMM EVOLUTION	11
4、 現金流量內插法.....	13
五、 實際範例.....	17

一、可贖回 CMS 連結每日區間計息債券

1、契約內容

可贖回CMS連結每日區間計息債券(Callable CMS-Linked Daily Range Accrual Note，以下簡稱CCDRA)。CCDRA為系列專案第六項產品，此產品係依據同系列專案第五項產品固定期限交換連結每日區間計息債券(CDRA，CMS-Linked Daily Range Accrual Note)附加債券贖回權利之結構型產品，即發行人於凍結期間外可支付債券贖回價格併提前終止契約。

本文件為元富證券債券部評價模型之說明文件，在符合實務市場下，計算CCDRA的理論價格，藉此了解商品的合理性與正確性。CCDRA為基於原始CDRA架構加上債券贖回權之產品，故CCDRA推導過程將引述CDRA技術文件作為此技術文件之參考。CCDRA產品連結CMS每日區間計息債券，根據市場上每日CMS利率指標是否介於預設區間決定債券支付票面利率。由於市場上CMS利率皆為較長天期利率指標，一般型利率模型較難描述長天期利率結構的特徵，故此文件運用LIBOR Market Model推展出遠期利率擴散過程(Forward Diffusion)，並透過最小平方蒙地卡羅法(Least Squares Monte Carol Method，以下簡稱LSMC法)以計算CCDRA產品之理論價格。

範例契約如下：

- (1) 債券發行人(Issuer): 元富證券股份有限公司
- (2) 商品別(Product): 固定期限交換連結每日區間計息債券
- (3) 本金(Principal Amount): USD 10,000,000.00
- (4) 計價幣別(Currency): USD
- (5) 生效日(Effective Date): 2022 年 3 月 31 日
- (6) 到期日(Maturity Date): 2027 年 3 月 31 日
- (7) 贖回價格(Call Price): 100.00
- (8) 凍結期間(Freeze): 1 年
- (9) 下限利率(Floor): 0.00%
- (10) 上限利率(Ceiling): 4.25%
- (11) 連結標的(Index): 2 年期固定期限交換利率(Bloomberg Ticker USSW2 Curncy)

(12) 票面利率(Coupon Rate): 固定利率 1.65%

(13) 付息頻率(Frequency): Quarterly

(14) 計息基礎(Daycount): 30/360

二、 LMM 利率模擬

過去傳統利率模型描繪瞬間短期利率或瞬間遠期利率的隨機過程，如 Vaiscek、Hull-White、HJM 等。由於這些利率是無法從真實市場觀察取得，，因此自 1997 年 Brace、Gatarek 及 Musiela(BGM)提出 LIBOR 市場模型(LIBOR Market Model)。此模型是根據市場上觀察得到的指標利率，而非抽象不存在的瞬間短期利率及瞬間遠期利率，所以透過市場資料即可描述目前的市場利率期間結構與利率波動結構，大幅增加產品定價的準確性。由於 CDRA 係以連結長天期 CMS 利率作為標的，因故對於其利率的模擬在使用 LMM 模型上能夠更加準確。

1、LMM 模型

假設我們有一時間序列， $0 = T_0 < T_1 < \dots < T_{n+1}$ ，令 $\delta_j = T_{j+1} - T_j$ ， $P(t, T)$ 表示到期於 T 的零息債券於 t 時的債券價格。為了簡化模型，假設所有 $\delta_j = \delta$ 。令 $L(t, T)$ 表示 T 至 $T + \delta$ 的遠期利率於 t 時的價格，根據無套利原則，

$$L(t, T) = \frac{\frac{P(t, T)}{P(t, T + \delta)} - 1}{\delta} ,$$

由於遠期利率 $L(t, T)$ 會於 T 時進行重設，在這之後將會成為定值。故為簡化標記，令 $L(t, T_j)$ 記為 $L_j(t)$ ，以及 $P(t, T_j)$ 記為 $P_j(t)$ 。

我們以 Spot LIBOR 測度空間作為模型實作測度，故在此選用 Money Market Account 作為計價單位，其於 t 時點的價值為

$$N(t) = P(t, T_{\eta(t)}) \prod_{i=1}^{\eta(t)-1} (1 + \delta L_i(T_i)) ,$$

其中 $\eta(t)$ 為一個唯一滿足 $T_{\eta(t)-1} \leq t < T_{\eta(t)}$ 的指標。此計價單位可視為一個到期於 T_1 之零息債券將其收益及本金不斷再投資於下一天期直至 $T_{\eta+1}$ 之投資組合。

基於 LMM 架構下，我們假設遠期利率服從隨機過程如下，

$$dL_i(t) = \mu_i(L(t), t)(L_i(t) + \alpha_i)dt + \sigma_i(t)(L_i(t) + \alpha_i)dW_i(t) ,$$

其中 $\sigma_i(t)$ 為已決定函數，而 α_i 為固定飄移常數， W_i 則為 Spot LIBOR martingale 風險測度下之隨機布朗運動。 $L(t)$ 表示 t 時點的遠期利率狀態變數，而 μ_i 函數則僅與無套利條件有關。假設任意 W_i 與 W_j 之間相關係數為 $\rho_{i,j}$ 。考量模型簡化以及 time-

homogeneity，在此簡化作 α ，並同時將 $L_j(t) + \alpha$ 簡化為 $\tilde{L}_j(t)$ 。由於折現因子於該風險測度下服從 martingale 性質，假設以 $N(t)$ 作為計價單位，在其等價測度下可推導飄移項

$$\mu_i(L(t), t) = \sum_{j=\eta(t)}^i \frac{\tilde{L}_j(t)\delta}{1 + L_j(t)\delta} \sigma_{i,j}(t)$$

其中 $\sigma_{i,j}(t) = \sigma_i(t)\sigma_j(t)\rho_{i,j}$ ， $\sigma_i(t)$ 為 $\tilde{L}_i(t)$ 之波動度。

考慮區間 (s, t) 遠期利率模擬，我們計算區間內遠期利率對數之共變異數矩陣，

$$\sigma_{i,j}(s, t) = \int_s^t \rho_{i,j} \sigma_i(r) \sigma_j(r) dr ,$$

對於共變異數矩陣我們可以將其作矩陣開方根處理，即找到矩陣 A 滿足

$$\sigma_{i,j} = \sum_{r=1}^F A_{i,r} A_{r,j}^t$$

最後我們可透過以上飄移項推導出遠期利率擴散函數，

$$\log(L_i(t)) = \log(L_i(s)) - \frac{1}{2} \sigma_{i,i}(s, t) + \sum_{r=1}^F A_{i,r} (e_{r,i} + Z_r)$$

三、 LMM 模型進行利率模擬

藉由上述章節推導，我們可透 LMM 模型模擬 CMS 利率擴散函數，並透過隨機項的大量生成模擬出無數條 CMS 利率路徑。由於債券贖回權屬於美式選擇權的一種，因此在進行利率路徑的模擬上無法使用傳統的蒙地卡羅模擬法，取而代之的為最小平方蒙地卡羅法 LSMC。其中 LSMC 之理論於此系列專案第一項產品技術文件已詳述之，故在此技術文件僅針對 LMM 模型下在 CCDRA 產品之實作情形做說明。

1、CCDRA 產品假設

實務上在進行長天期的利率模擬考量效率性等因素，通常會使用各種近似方法以簡化模型。因此針對此產品設計上，我們將採用 M. Joshi 透過改良 Schlogl 內插法來進行 CCDRA 之評價。

考慮區間計息產品每一計息區間終點 $T_i (i \geq 2)$ 需支付區間計息 S_i ，其區間計息可表示如下，

$$S_i = \frac{Q_i}{n_i} \sum_{j=1}^{n_i} I_{B < R(T_{i-1} + t_j^{i-1}) < U} ,$$

其中 n_i 表示第 $(i-1)$ 之計息區間日數， $R(t)$ 表示參考利率於 t 時點的利率， t_j^{i-1} 則表示時點 T_{i-1} 至第 j 個計息日期間間隔。在實務上 Q_i 一般可為隨機的，而 B_i 及 U_i 一般情況為給定函數(在此設為定值)。

在 CDRA 中產品情境令 $R(t) = SR_{t,x}(t)$ ， $Q_i = q$ ，即參考利率為 x 天期之 CMS 利率，並令 Q_i 乘數為定值 q ，起始於 t 時點之區間計息產品。為簡化後續模型推導，令 $\mathbb{E}_T^t(\cdot) = \mathbb{E}_T(\cdot | \mathcal{F}_t)$ ，其中上標 t 對應於以 $P(t, T)$ 為計價單位之 martingale 測度；而若無上標，則表示對應於以 $N(t)$ 為計價單位之 martingale 測度。另外，令 $\theta(t) = T_{\eta(t)} - t$ 以及 $\vartheta(t) = t - T_{\eta'(t)}$ 表示時點 t 至下一計息日以及前一計息日之前間隔，其中 $\eta'(t) = \eta(t) - 1$ 。

由於此次實作產品為每日區間計息產品，為簡化模型以及後續模型架構滿足一致性，在此我們將計息區間由每日計息分割為較寬期間頻率之區間集合，即 $T_0 < T_1 < \dots < T_{n+1}$ 。

2、區間計息內插法

透過簡化後的 LMM 模型對於模擬出之路徑可產生出對應 $T_{\eta(t)}$ 至 T_{n+1} 時點之 CMS 利率 $SR_{t,x}(s)$ ，然而在 CDRA 產品評價中，我們應考量每日區間計息之可能性，因此我可透過簡單內插法估算模擬出之利率路徑落入計息區間之可能性。

令 $R_H = \max(R(T_{i-1}), R(T_i))$ ， $R_L = \min(R(T_{i-1}), R(T_i))$ ，則我們有 CMS 利率落於區間內之比率 $Ratio_i$ 如下，

$$Ratio_i = \begin{cases} 1, & B \leq R_L \leq R_H \leq U \\ 0, & R_H \leq B \text{ or } R_L \geq U \\ \frac{\min(U, R_H) - \max(B, R_L)}{R_H - R_L}, & \text{else} \end{cases}$$

3、最小平方蒙地卡羅法(Least Square Monte Carlo Simulation Method)

蒙地卡羅模擬法乃是使用機率上的大數法則，針對估計對象中的隨機變數，利用多次反覆的模擬實驗，從而得到估計對象的可能結果。然而傳統蒙地卡羅法僅能針對

歐式選擇權之金融產品進行定價，無法評價提前履約之商品損益。因此在評價包含美式或是亞洲式選擇權之金融商品時，需要應用不同統計方法以估計模擬路徑下最佳行權時點。在此產品中即應用最小平方蒙地卡羅法，透過上述章節所推導出之產品價格模擬路徑進一步估算出每條價格路徑下的最佳行權時間點，進而推算評價時間點之產品理論價格。

在此文件中欲評價之 CCDRA 價格，其付息金額是根據契約期間內參考利率落入計息區間日數作為判斷依據，含有路徑相關的特性，因此透過 LMM 模型能夠描繪出契約期間內不同時點之參考利率對應之商品價格，並依此判斷 CCDRA 現金流。實作流程如下：

- (1) 利用上述章節所推導出之遠期利率擴散方程式可展出數條 LMM 模型下之遠期利率路徑 $L_i^n(t)$ 。
- (2) 透過模擬出之遠期利率，我們可計算出 CMS 利率，

$$\widehat{SR}_{t,x}^n(s) = \sum_{j=0}^{x-1} w_j \widehat{L}_j^n(s + j\delta)$$

- (3) 藉由區間計息內插法估計出每一條路徑下 CMS 利率落入計息區間之計息天數以及 CCDRA 於任意行權時點 T_K 之履約損益，

$$tau_{t,j} = Ratio_{t,j} \times \delta$$

$$Payoff_{t,j} = N \times \left(1 + \sum_{j=0}^{T_K} q \times tau_{t,j} \right)$$

- (4) 利用 LSMC 法求得第 j 條路徑在時間點 t 時的 CCDRA 條件期望值 $E[CCDRA]_{t,j}$ ，即繼續持有 CCDRA 的價值。

$$E[CCDRA]_{t,j} = E[CCDRA_{t,j} | \widehat{SR}_{t,x}^n]$$

其中 $\widehat{SR}_{t,x}^n$ 為第 j 條路徑在時間點 t 時的 CMS 利率， $CCDRA_{t,j}$ 為相對於在時間點 t 時未履約的 CCDRA 價值。

- (5) 求算出時間點 t 時 j 路徑的選擇權期望繼續持有價值 $E[CCDRA]_{t,j}$ ，並與提前贖回價 $Call$ 取其最大值為 t 時點 j 路徑的 CCDRA 理論價值。
- (6) 回到(4)，直到 $t = 0$ 即為評價時的理論價值。
- (7) 最後把所有路徑在 $t = 0$ 時的理論價值加總平均即為 CCDRA 之理論價值。

四、 Quant Lib 實作程式流程說明

本產品之程式實作說明，鑒於 Quant Lib 程式庫(以下簡稱 QL)專案之部分物件，於公司內部已有相關產品之技術文件逕行說明，故在此僅以此產品評價中非源生於 QL 內部物件，抑或是過去產品為實作過之物件進行說明。另外，評價流程部分則採第一章節之第 1 小節之範例契約作為實際範例。

1、 利率期間結構實作

利率期間曲線實作可參考【零息債可贖回債券技術文件】之利率期間曲線實作。此程式之曲線建構由 C# 讀取外部 csv 文件，文件格式固定為 Bloomberg 輸出，若欲手動輸入則可於程式內部給定。

(1) 建構利率期間結構曲線

輸入評價日及利率檔路徑，讀取外部利率資料檔，並透過 Bootstrapping 建立利率交換曲線。

```
public void SetupTermStructure()
{
    Settings.setEvaluationDate(Today);

    //----- Create Original Curve -----
    dataReading = new DataReading();
    dataReading.SetupTermStructure(Today, swap_path);
    HTS = dataReading.forwarding;
```

2、 LMM 模型參數 Calibration

在 QL 程式庫裡針對不同模型，皆已建立完整的 Calibrate 模組。因此我們僅需將對應模型實作後，將市場上 Swaption 之 Volatility 報價作為校準資料，即可以此作為本次 LMM 模型之參數校準

(1) Swaption 市場資料

輸入 Swaption 資料路徑，讀取外部利率資料檔，並建立 Swaption 結構向量供後續市場校準使用。

```
public List<DataReading.SwptDatum> listSwaption;
public void ReadSwaptionVol()
{
    // Read Non-Shifted Swaption Vol
    dataReading.SetupSwaptionMat(swaption_path);
    listSwaption = dataReading.swaptionMat;
}
```

```
public List<SwptDatum> SwaptionMat(string swaption_path)
{
    fetching_datas(swaption_path);
    List<SwptDatum> swpts = new List<SwptDatum>();
    for (int i = 0; i < swaption_Mat.Count; i++)
    {
        for (int j = 0; j < swaption_Len.Count; j++)
        {
            if (swaption_Vols[i * swaption_Len.Count + j] > 0)
            {
                swpts.Add(new SwptDatum(
                    swaption_Mat[i],
                    swaption_Len[j],
                    swaption_Vols[i * swaption_Len.Count + j]));
            }
        }
    }
    return swpts;
}
```

(2) Libor Forward Model Process 物件

實作 Libor Forward Model Process 物件需輸入參數惟 LMM 模型欲模擬遠期利率期數(size)，以及模擬利率指標(Index)。

- i. size 為模擬利率個數，對於結構型產品之模擬需於契約到期後額外模擬一期利率作為未來現金流折現使用。
- ii. Index 為 LMM 利率指標，以契約標的 CMS 利率所參照之指標利率為基準，此範例契約為 Libor 3M。

(3) LMM Volatility Model 物件

此專案實作 LMM Volatility Model 物件以 QL 內部程式 Lm Ext Linear Exponential Vol Model(又稱 ABCD 波動度模型)實作之，其參數為 LMM 模型待校準參數之一，於市場校準環節僅需輸入 LMM Process 模擬利率時間點(Fixing Times)及任意四常數分別對應模型內部四項參數(a、b、c、d)即可。

(4) LMM Correlation Model

此專案實作 LMM Correlation Model 物件以 QL 內部程式 Lm Linear Exponential Correlation Model 實作之，其參數為 LMM 模型待校準參數之一，於市場校準環節需輸入模擬利率個數(size)以及對應模型內部兩參數(rho、beta)即可。

(5) Libor Forward Model

Libor Forward Model 為 LMM 模型物件主體，透過上述物件堆疊可實作之，其參數分別輸入上列 Process、LMM Volatility Model 及 LMM Correlation Model 即可實作之。

(6) 實作 Pricing Engine 物件

- i. Model(模型假設)，即評價金融商品時所使用之假設模型，此處由於我們假設利率服從 LMM 模型，故我們以上一小節實作出之 LMM 模型作為參數代入。
- ii. Engine(評價引擎)，在此採用 QL 程式庫內建 LMM 模型 Swaption 解析解，LFM Swaption Engine 作為評價引擎。
- iii. Set Pricing Engine(評價引擎設定)，即選定該商品評價模型，此處採用上一小節所實作出之 Engine。

(7) Calibration

- i. 藉由(1)所輸入之 Swaption 市場波動度，建立足夠數量之 Swaption Helper 作為市場校準物件，並透過清單物件集合上述 Swaption Helper 物件。
- ii. 對應上述每一 Swaption Helper 物件設定其 Pricing Engine。
- iii. 對已實作 Libor Forward Model 實作 calibrate 方法。
- iv. 輸出 Model 參數，即可得 LMM 模型市場校準參數。

```
int size = evldates.Count;
IborIndex Index = new USDLibor3M(HTS);

LiborForwardModelProcess process = new LiborForwardModelProcess(size, Index);
// Use Shifted TS
Handle<YieldTermStructure> termStructure = Index.forwardingTermStructure();

// set-up the model
LmVolatilityModel volaModel = new LmExtLinearExponentialVolModel(
    process.fixingTimes(), 0.5, 0.6, 0.1, 0.1);

LmCorrelationModel corrModel = new LmLinearExponentialCorrelationModel(size, 0.5, 0.8);
LiborForwardModel model = new LiborForwardModel(process, volaModel, corrModel);

// set-up calibration helper
List<CalibrationHelper> calibrationHelper = new List<CalibrationHelper>();

for (int j = 0; j < listSwaption.Count; j++)
{
    Handle<Quote> swaptionVol = new Handle<Quote>(new SimpleQuote(listSwaption[j].vol));
    CalibrationHelper swaptionHelper = new SwaptionHelper(listSwaption[j].matlen, listSwaption[j].swaplen,
        swaptionVol, Index, new Period(6, TimeUnit.Months), new Thirty360(),
        Index.dayCounter(), termStructure, true);
    swaptionHelper.setPricingEngine(
        new LfmSwaptionEngine<LiborForwardModel>(model, termStructure));
    calibrationHelper.Add(swaptionHelper);
}

LevenbergMarquardt om = new LevenbergMarquardt(1e-6, 1e-6, 1e-6);
model.calibrate(calibrationHelper, om, new EndCriteria(
    200, 100, 1e-6, 1e-6, 1e-6));
```

3、LMM Evolution

根據前述章節之 LMM 利率模型推導，我們可實作出 LMM 模型

遠期利率擴散函數，

$$\log(L_i(t)) = \log(L_i(s)) - \frac{1}{2} \sigma_{i,i}(s, t) + \sum_{r=1}^F A_{i,r} (e_{r,i} + Z_r) ,$$

由於上述遠期利率擴散函數涉及變數間疊代關係，若依循傳統利率刻劃方式將過於繁瑣即耗時。因此實作上參考 QL 架構之遠期利率模擬方法，故在此技術文件即不另外進行實作流程說明，僅以此架構之概念輔佐上述章節所推導之變數意義作為說明。

(1) LMM 模型共變異數結構

利率擴散函數需先計算不同期間之遠期利率共變異數 $C_{i,j}(s, t)$ ，其中共變異數可透過上述章節之波動度模型以即相關係數模型所計算出之波動度結構以及相關係數所計算之。

```
// One Correlation Matrix per evolution step, every step equal
G CORR = Algorithm.ExpCorr(rateTimes, LTCorr, beta, 1.0);
for (int i = step; i < NRates; ++i)
{
    for (int j = i; j < NRates; ++j)
    {
        cov = Algorithm.ABCDVolCov(a, b, c, d, effStartTime, effStopTime,
            rateTimes[i], rateTimes[j]);
        cov = adjK[i] * adjK[j] * cov;

        GCDV[i, j] += cov * G CORR[i, j];
    }
}
```

(2) LMM 模型飄移項

LMM 模型飄移項根據上述章節推導可表示如下，

$$\mu_i(L(t), t) = \sum_{j=\eta(t)}^i \frac{\tilde{L}_j(t)\delta}{1 + L_j(t)\delta} \sigma_{i,j}(t)$$

```
for (int i = step; i < NRates; ++i)
    tmp1[i] = (currentFwds[i]) / (oneOverTaus[i] + currentFwds[i]);

for (int i = step; i < NRates; ++i)
{
    drifts1[i] = 0.0;
    for (int j = step; j < i + 1; j++)
    {
        drifts1[i] += tmp1[j] * COV[i, j];
    }
}
```

(3) LMM 隨機項

LMM 隨機項係由共變異數結構方根因子 A 以及隨機布朗運動所構成，其表示如下，

$$-\frac{1}{2}\sigma_{i,i}(s,t) + \sum_{r=1}^F A_{i,r}(e_{r,i} + Z_r),$$

(4) LMM 模擬利率

透過上述飄移項及隨機項拆解以及疊代，可計算不同期間之 LMM 模型下對數常態遠期利率之模擬路徑，故欲得到 LMM 模型下之遠期利率模擬路徑僅需對前者取指數即可得之。

```
for (int i = step; i < NRates; ++i)
{
    logFwds[i] += drifts1[i] + fixedDrifts[i];
    logFwds[i] += Vector.DotProduct(A.row(i), brownians);

    if (logFwds[i] < MaxLogFwd)
    {
        nextFwds[i] = Math.Exp(logFwds[i]);
    }
    else
    {
        nextFwds[i] = Math.Exp(MaxLogFwd);
    }

    //nextFwds[i] = Math.Exp(logFwds[i]);
}
currentFwds = new Vector(nextFwds);
```

4、現金流量內插法

根據模擬出之 Libor 3M 利率曲線，我們可以推導出對應之 CMS 利率路徑。並依據上一章節所推導出之利率內插過程計算出每一期間內，CMS 利率落於利率區間之比例。

CCDRA 之現金流相當於契約落於利率區間天數乘上付息利率，因此透過上述所計算出之每一期間之落入區間比例，我們可計算出每一條路徑下 CMS 落於區間之天數，並且依據該天數計算對應路徑下之現金流量。

(1) CMS 利率推導

根據利率交換公式定義，我們可透過拆解利率交換固定端及浮動端以計算 CMS 利率如下。

```
#region Parallel_Interpolate
for (int p = 0; p < NPath; p++)
{
    Vector QCMS = new Vector(MSteps);
    Vector FloatQ = new Vector(MSteps);
    Vector FixedSA = new Vector(MSteps);
    QCMS[0] = CmsQuote;
    #region Interval_others
    for (int k = 1; k < MSteps; k++)
    {
        Vector CurrentFwdRate = new Vector();
        for (int j = k; j < k + swapLens; j++)
        {
            CurrentFwdRate.Add(LFwdMatrix[p][k, j]);
        }

        Vector CurrentDF = new Vector();
        CurrentDF.Add(1.0);
        for (int j = 1; j < swapLens + 1; j++)
        {
            double df = CurrentDF[j - 1] / (1 + CurrentFwdRate[j - 1] * accruals[j]);
            CurrentDF.Add(df);
        }

        for (int j = 0; j < swapLens / flostrateNums; j++)
        {
            double CF = Nominal * CurrentFwdRate[j] * accruals[k + flostrateNums * j];
            FloatQ[k] += CF * CurrentDF[flostrateNums * j + 1];
        }
        for (int j = 0; j < swapLens / fixrateNums; j++)
        {
            double CF = Nominal * accruals[k + fixrateNums * j];
            FixedSA[k] += CF * CurrentDF[fixrateNums * j + 1];
        }

        double CMS = FloatQ[k] / FixedSA[k];
        QCMS[k] = CMS;
    }
    LQCMS.Add(QCMS);
}
#endregion
```

(2) 利率內插法

透過上述模擬出之 CMS 利率路徑，計算 CMS 利率於不同期間落於利率區間之比例。

```
#region Parallel_CFAAnalysis
for (int p = 0; p < NPath; p++)
{
    Vector QCMS = LQCMS[p];
    Vector Ratio = new Vector(MSteps);

    for (int j = 0; j < MSteps - 1; j++)
    {
        double cms_l = Math.Min(QCMS[j], QCMS[j + 1]);
        double cms_h = Math.Max(QCMS[j], QCMS[j + 1]);
        if (cms_l >= B && cms_h <= U)
            Ratio[j] = 1.0;
        else if (cms_h < B || cms_l > U)
            Ratio[j] = 0.0;
        else
            Ratio[j] = (Math.Min(U, cms_h) - Math.Max(B, cms_l)) / (cms_h - cms_l);
    }
    LCouponRatio.Add(Ratio);
}
```

(3) 現金流量內插法

透過上述所得天數比例，我們可計算出不同路徑下每一期間之現金流量，並可計算出對應路徑下之現金流量。

```
Vector CF = new Vector(MSteps);
if (Today != cfdates[InitPoint])
    CF[0] = Nominal * Coupon * InRange / (Today - cfdates[InitPoint]) *
        new SimpleDayCounter().yearFraction(cfdates[InitPoint], Today);
for (int i = 0; i < MSteps; i++)
{
    CF[i] += Nominal * Coupon * Ratio[i] * accruals[i];
}
CF[MSteps - 1] += Nominal;
LCF.Add(CF);

double NPV = 0;
for (int i = 0; i < MSteps; i++)
{
    NPV += CF[i] * VDisc[i];
}
MCF.setRow(p, CF);
MCMS.setRow(p, QCMS);
LNPV.Add(NPV);
```

(4) 最小平方蒙地卡羅法

根據上一小節所得產品現金流量於不同路徑下的節點，根據最小平方蒙地卡羅法可以由到期時點回推產品於現金流節點下的期望值與繼續持有價值，並且疊代出其債券價格。

```

for (int j = MSteps - 1; j >= 0; j--)
{
    List<int> RegPath;
    int OBS;

    RegPath = new List<int>(0);
    for (int i = 0; i < NPath; i++)
    {
        HoldValue[i, j] = j == MSteps - 1 ? MCF[i, j] : Valuc[i, j + 1] * VDisc[j + 1] / VDisc[j] - MCF[i, j];
        Valuc[i, j] = HoldValue[i, j];
        if (HoldValue[i, j] < -1 * (CallValue[j] + MCF[i, j]) && j + InitPoint > frecczo) // Exercise better -- ITM
            RegPath.Add(i);
    }

    OBS = RegPath.Count;
    Matrix MY = new Matrix(OBS, 1);
    Matrix MX = new Matrix(OBS, 4);
    for (int i = 0; i < OBS; i++)
    {
        int obs = RegPath[i];
        MY[i, 0] = Valuc[obs, j + 1] * VDisc[j + 1] / VDisc[j] - MCF[i, j];

        double ar = MCMS[obs, j];
        MX[i, 0] = 1.0;
        MX[i, 1] = ar;
        MX[i, 2] = ar * ar;
        MX[i, 3] = ar * ar * ar;
    }

    Matrix MXT = new Matrix(Matrix.transpose(MX));
    Matrix MA = new Matrix(MXT * MX);
    Matrix MAI = new Matrix(Matrix.inverse(MA));
    Matrix MC = new Matrix(MXT * MY);
    Matrix MB = new Matrix(MAI * MC);

    for (int i = 0; i < OBS; i++)
    {
        int obs = RegPath[i];
        double ar = MCMS[i, j];

        double Expect = MB[0, 0] * 1.0
            + MB[1, 0] * ar + MB[2, 0] * ar * ar + MB[3, 0] * ar * ar * ar;

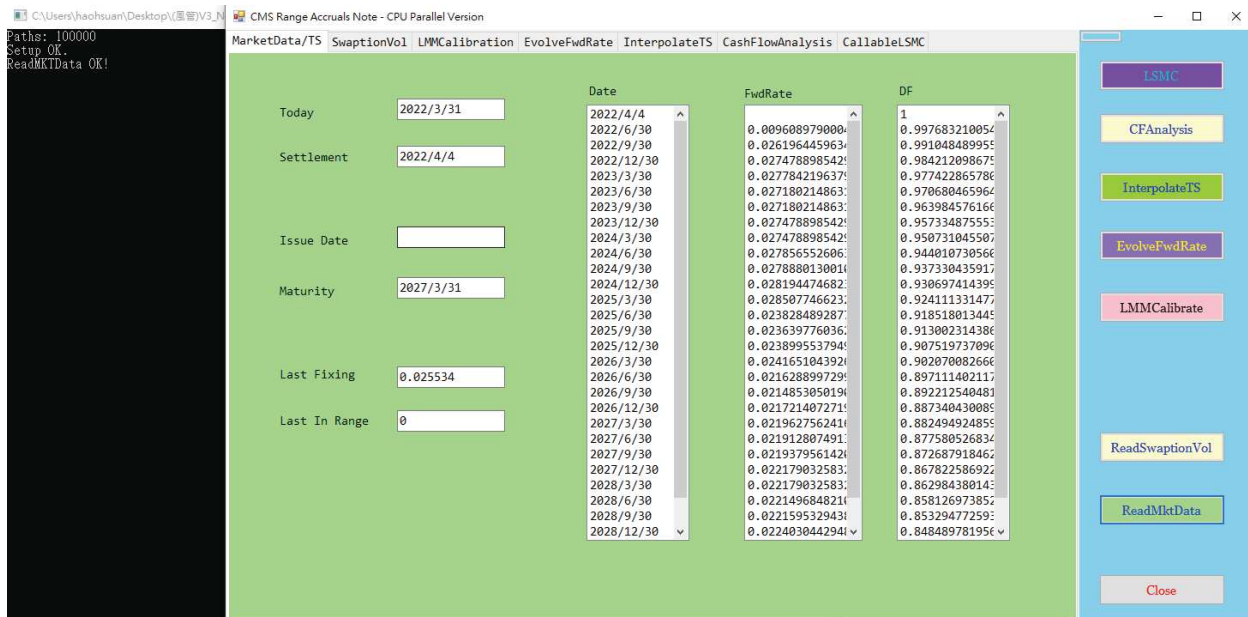
        if (Expect < -1 * (CallValue[j] + MCF[obs, j]))
        {
            Valuc[obs, j] = -1 * (CallValue[j] + MCF[obs, j]);
            calltime[obs] = j;
        }
    }
}

```

五、 實際範例

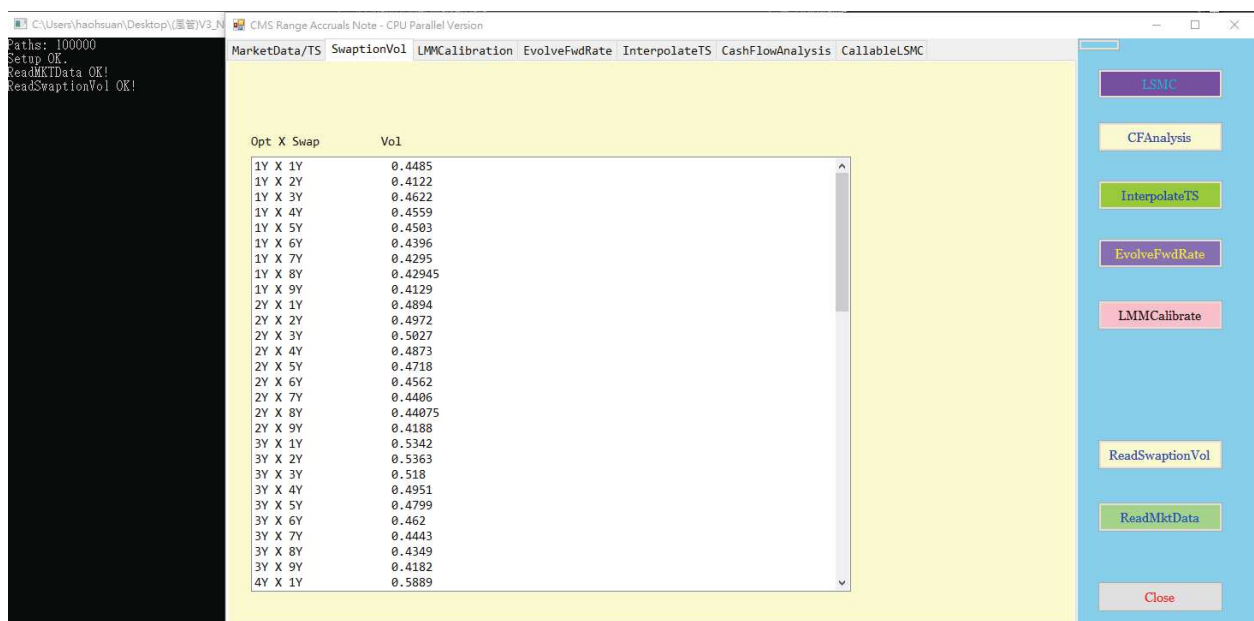
(1) 設定資料結構

於程式內部設定契約基本資料後，由程式讀取市場資料(Deposit Data & Swap Data)。



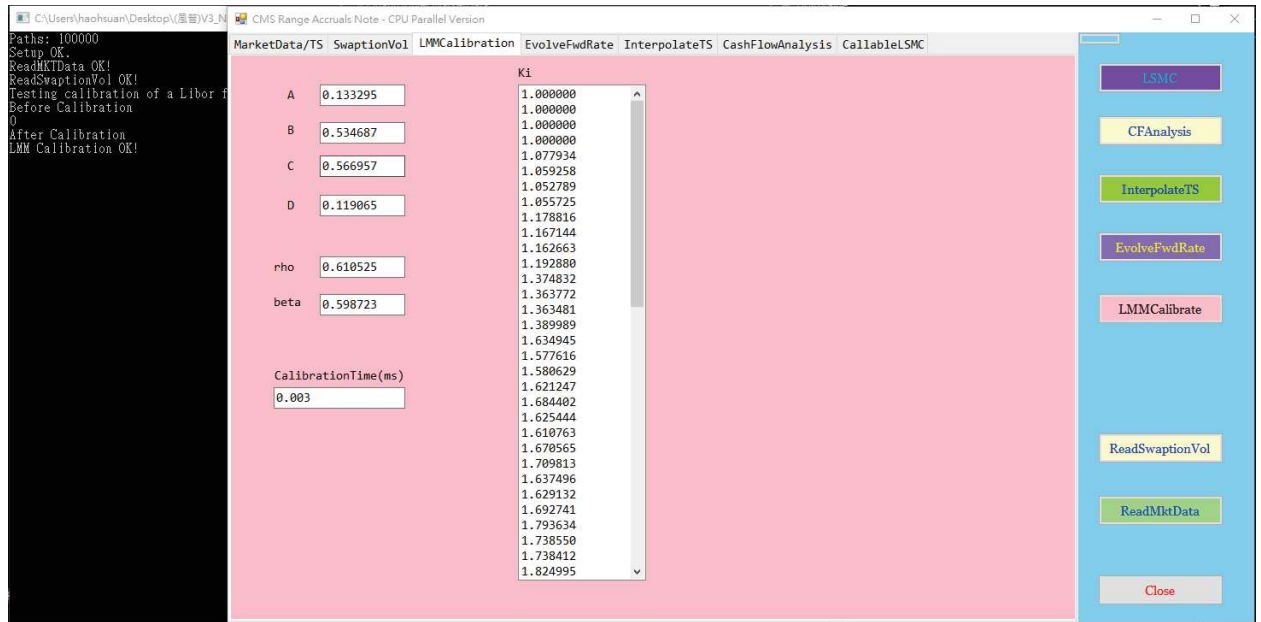
(2) 市場波動度讀取

由程式讀取市場利率交換選擇權資料(SwapVol)。



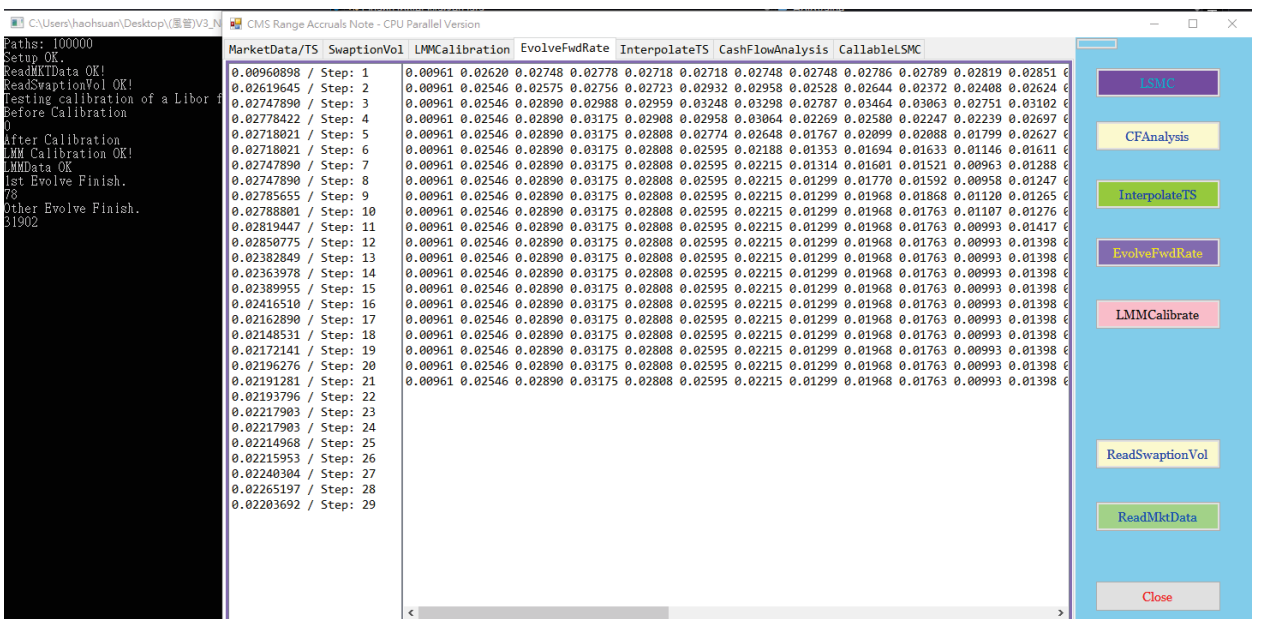
(3) LMM 模型市場校準

透過 LMM 模型執行市場校準，並記錄其模型參數。



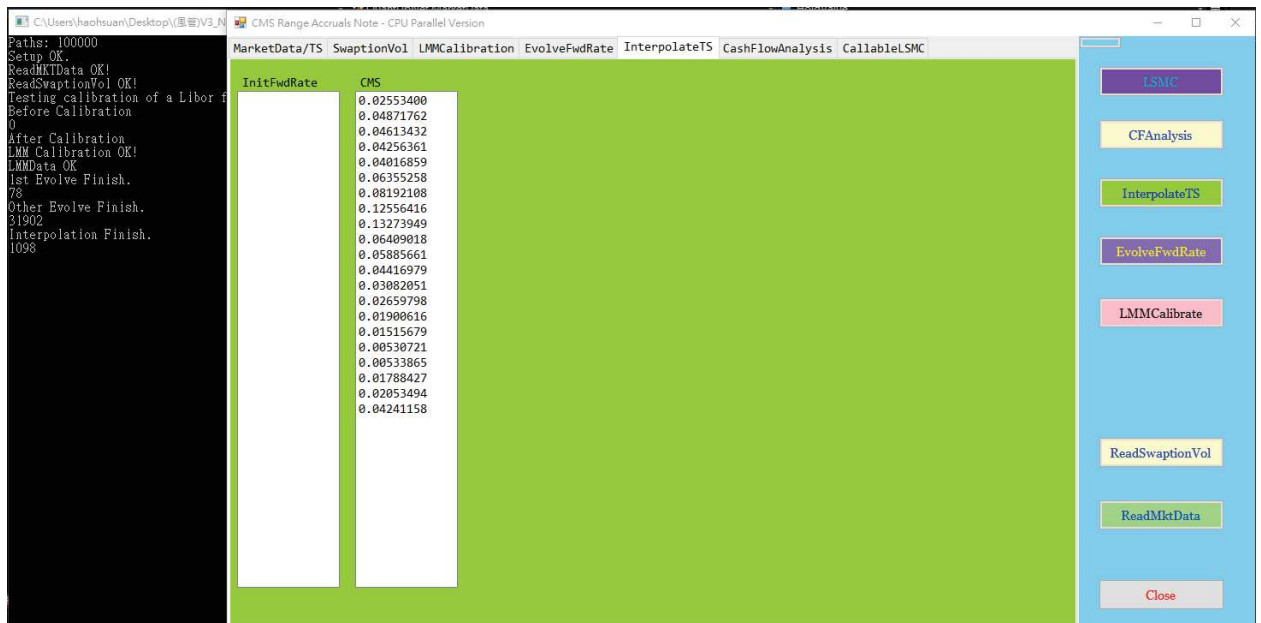
(4) LMM 利率模擬

透過上述市場校準參數模擬出遠期利率路徑。



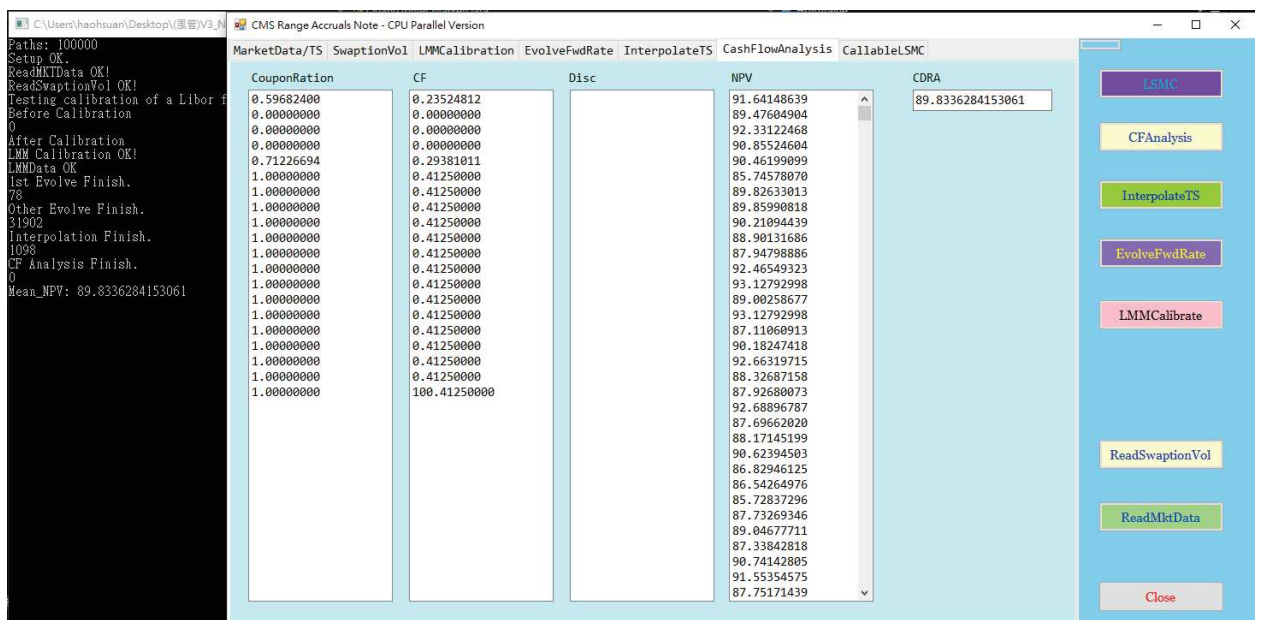
(5) CMS 利率推導

依據不同利率路徑計算每條路徑對應之 CMS 利率路徑。



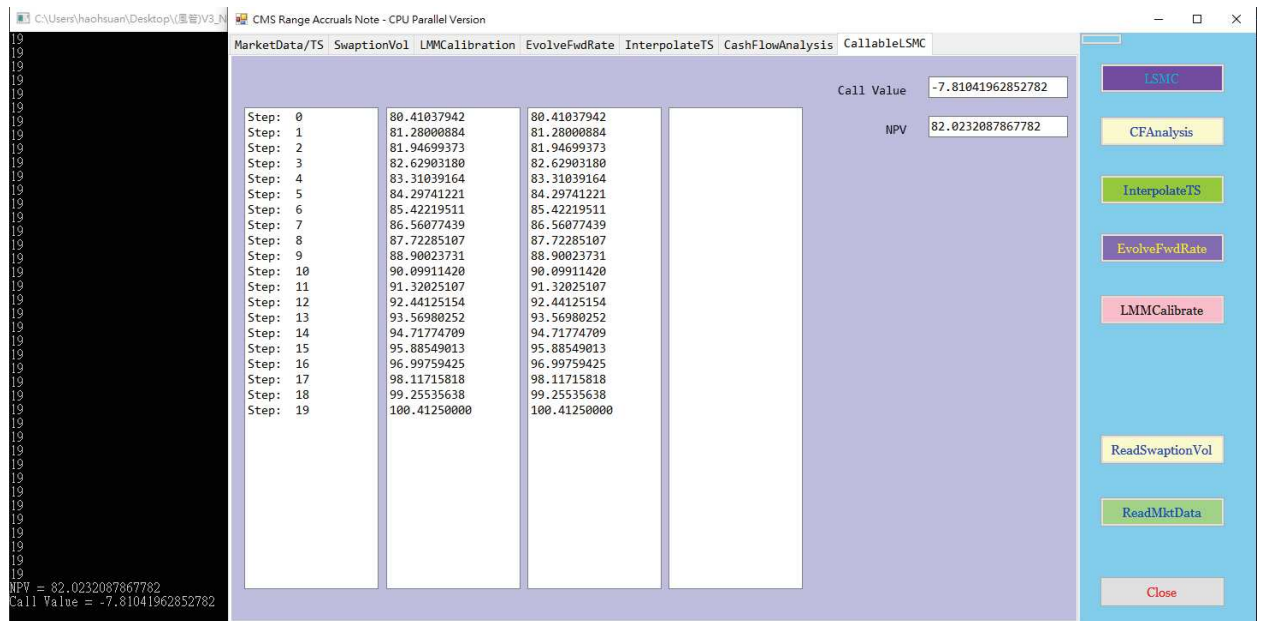
(6) CF Analysis

計算 CDRA 契約現金流及不含贖回權理論價格。



(7) LSMC

透過 LSMC 法計算 CCDRA 理論價格。



● 計算範例

假設發行於 2022 年 3 月 31 日，發行期間 5 年，凍結期間 1 年，到期日為 2027 年 3 月 31 日，指標利率為 2 年期 CMS 利率之每日區間計息產品，契約條件如第一章節所述，範例契約參數如下，假設評價日為 2022 年 3 月 31 日，假設指標利率距上一次付息期間內有 0 日落入計息區間：

	參數代號	參數名稱	參數內容
輸入參數	Today	評價日	2022/3/31
	Issue Date	發行日	2022/3/31
	Maturity	到期日	2027/3/31
	Nominal	名目本金	100
	Coupon	付息利率	1.65%
	B/U	上/下限利率	0.0 % / 4.25 %
	Last fixing	指標利率最報價	2.55 %
	In Days	上一落入區間數	0
	Credit spread	信用貼水	0.5 %

	Swap_curve	Swap 曲線資料	Term	InstType	Bid	Ask	Mid
			3 MO	CASH	0.96157	0.96157	0.96157
			2 YR	SWAP	2.5498	2.557	2.5534
			3 YR	SWAP	2.64612	2.65998	2.65305
			4 YR	SWAP	2.59307	2.60253	2.5978
			5 YR	SWAP	2.51706	2.52864	2.52285
			6 YR	SWAP	2.47466	2.48534	2.48
			7 YR	SWAP	2.44803	2.45657	2.4523
			8 YR	SWAP	2.42637	2.43363	2.43
			9 YR	SWAP	2.41064	2.41816	2.4144
			10 YR	SWAP	2.40258	2.41042	2.4065
			11 YR	SWAP	2.40339	2.41261	2.408
			12 YR	SWAP	2.40083	2.40718	2.404005
			15 YR	SWAP	2.39665	2.40265	2.39965
			20 YR	SWAP	2.37629	2.38551	2.3809
			25 YR	SWAP	2.31452	2.32247	2.318495
			30 YR	SWAP	2.24843	2.25747	2.25295
			40 YR	SWAP	2.08358	2.09043	2.087005
			50 YR	SWAP	1.91373	1.94511	1.92942
輸出參數	CCDRA	CCDRA Value	82.0232087867782				